Autoscaling proactif en microservices

Master 2 Informatique
Université de Yaoundé I

24 octobre 2026

Q1. Qu'est-ce que l'HPA de Kubernetes ?

Le *Horizontal Pod Autoscaler* est le contrôleur d'autoscaling horizontal natif de Kubernetes. Il ajuste le nombre de réplicas d'un déploiement selon l'algorithme

$$\text{desiredReplicas} = \left\lceil \text{currentReplicas} \times \frac{\text{currentMetricValue}}{\text{desiredMetricValue}} \right\rceil,$$

avec une tolérance par défaut de 10 % et une fenêtre de stabilisation anti-oscillation. Le HPA v1 est fondé sur le CPU ; le HPA v2 accepte des métriques personnalisées et externes et introduit le champ `behavior`. Il est fondamentalement **réactif et local** : il ne voit qu'un déploiement isolé, ce qui le rend vulnérable aux limites L1 (dépendances ignorées) et L2 (réactivité tardive).

Q2. C'est quoi le SLO ?

Le SLO (*Service Level Objective*) est la cible interne d'un triplet : le **SLI** est la mesure $\text{SLI}(T) = |\{e \in \mathcal{E}(T) : \text{good}(e)\}| \,/\, |\mathcal{E}(T)|$, le **SLO** est le seuil $\text{SLI}(T) \geq \rho$, et le **SLA** est l'engagement contractuel externe, plus laxiste (on vise 99,9 % en interne pour garantir 99,5 % contractuellement). Dans le problème d'optimisation, le SLO joue le rôle de *contrainte* tandis que le coût est la fonction objectif. Rigoureusement, il s'écrit sous forme probabiliste $\mathbb{P}[L(e) \leq \ell] \geq \rho$ ou quantile $Q_{1-\rho}(L(T)) \leq \ell$, et non de façon ponctuelle $\forall t,\ L(t) \leq \text{SLO}$ sur une variable aléatoire. Le budget d'erreur $1 - \rho$ lui donne son sens opérationnel.

Q3. Pourquoi ces seuils sont fixes au niveau de l'HPA de Kubernetes ?

Parce que le HPA est conçu comme un régulateur réactif et local : un point de consigne stable est nécessaire à la stabilité d'une boucle de rétroaction, et la tolérance de ± 10 % est un garde-fou anti-oscillation. Trois conséquences en découlent : inadéquation aux charges hétérogènes, dépendance à la valeur des `requests` (le seuil s'exprime en pourcentage des `requests`, pas de la consommation réelle, si bien qu'abaisser une `request` fait artificiellement exploser l'utilisation affichée), et réactivité structurellement tardive (délai de 90 à 150 secondes avec *cold start*). Les évolutions récentes (v2, champ `behavior`, tolérance configurable en v1.33 alpha) permettent de *régler* le seuil mais non de le rendre dynamique en fonction de la charge future ou des dépendances.

Q4. Existe-t-il des alternatives ou d'autres solutions permettant de faire de l'autoscaling ?

Oui. **KEDA** alimente le HPA en métriques externes et permet le *scale-to-zero*, mais hérite de la boucle réactive. Le **VPA** ajuste verticalement les `requests`/`limits`, pas le nombre d'instances. Les approches **prédictives** (ARIMA, LSTM, ST-GNN) anticipent mais restent souvent service par service ; les ST-GNN exploitent la topologie mais exigent des traces distribuées et sont opaques. L'**apprentissage par renforcement** (FIRM, Sinan) est global et proactif mais coûteux et fragile. La **théorie des files** (Jackson, Galileo) garantit mais sous hypothèses fortes. Les approches **décentralisées multi-agents** évitent le point de défaillance central mais peinent à prouver la convergence. Aucune ne coche simultanément L1 et L2 de façon industrialisable.

Q5. Pourquoi se fait-il de manière isolée sans gérer les dépendances entre microservices ?

Parce que le HPA est un régulateur local par conception, cohérent avec le dogme microservices du scaling indépendant : chaque service a son propre autoscaler, qui ignore l'état des autres. Techniquement, le HPA ne dispose d'aucun modèle de propagation sur le graphe d'appels : il n'observe que des *symptômes* locaux (CPU, mémoire) et non les *causes* amont. La littérature chiffre le coût de cette cécité : jusqu'à 43 % d'amélioration de performance avec un autoscaler conscient de la chaîne complète, et des violations de SLO transitoires systématiques pendant le délai de réaction.

Q6. Pourquoi vouloir optimiser globalement ?

Parce que la contrainte $L(n) \leq \text{SLO}$ n'est pas séparable : $\mathcal{F} = \{n : L(n) \leq \text{SLO}\} \neq \prod_i \mathcal{F}_i$, donc

$$\min_{n \in \mathcal{F}} C(n) \;\neq\; \sum_{i \in S} \min_{n_i \in \mathcal{F}_i} c_i n_i.$$

Un exemple chiffré sur la chaîne $A \to B$ montre un surcoût de 12 à 16 % pour un découpage local. À l'optimum, les coûts marginaux de réduction de latence s'égalisent : $\frac{1}{c_i} |\partial L_i / \partial n_i| = \nu$ pour tout i — condition que le HPA, avec sa cible fixe, ne peut structurellement pas vérifier. S'y ajoutent le couplage par propagation, le budget partagé, l'effet goulot, le *cold start* simultané et les phénomènes globaux (cascade, oscillation). Les contre-arguments à assumer sont la NP-difficulté, la sensibilité à l'erreur de modèle et le risque de point de défaillance central — d'où une **décomposition lagrangienne** où un prix scalaire ν est calculé et diffusé, faisant du HPA un autoscaler « à prix fixe arbitraire ».

Q7. Est-il possible d'intégrer un mécanisme de prédiction pour gérer la réactivité de l'HPA de Kubernetes ?

Oui, par trois points d'insertion. *(i) Métrique externe* : un adaptateur (KEDA avec Predict-Kube, `prometheus-adapter`) expose $\hat{\lambda}(t + \Delta)$ comme métrique au HPA v2. *(ii) Contrôleur prédictif autonome* (InferScale, contrôleurs LSTM) qui déplace la décision en amont du seuil. *(iii) Architecture hybride proactif/réactif* (AHPA d'Alibaba, AgileSphere) où le chemin réactif borne l'erreur de prédiction. Limites persistantes : la prédiction ne ferme pas L1 (elle reste service par service), l'incertitude de prédiction doit être quantifiée, le *cold start* Δ_i doit être intégré à la condition de déclenchement, et le *concept drift* impose une maintenance continue.

Q8. Qu'est-ce que Kubernetes, cluster ?

Kubernetes est la plateforme d'orchestration qui maintient l'état réel conforme à l'état désiré par une boucle de contrôle. Un **cluster** est l'infrastructure concrète : un *plan de contrôle* (`kube-apiserver`, `etcd`, `kube-scheduler`, `kube-controller-manager` où réside le HPA) et des *nœuds de travail* (`kubelet`, `kube-proxy`, runtime de conteneurs). Le HPA s'exécute dans le plan de contrôle central mais observe déploiement par déploiement : Kubernetes n'expose pas nativement la topologie des flux comme objet de premier ordre, d'où l'écart entre le graphe réel et sa représentation par le plan de contrôle.

Q9. Est-ce que le monitoring et la collecte des données devront être faits ?

Oui, c'est une condition de possibilité : sans traces, pas de α_{ij} ; sans séries temporelles, pas de prédiction ; sans métriques de latence, pas de vérification du SLO. Trois familles sont nécessaires : les **traces distribuées** (seule source des α_{ij}, via les relations parent-enfant des *spans*), les **métriques** de trafic, de latence (p95, p99) et de saturation (la profondeur de file étant plus prédictive que le CPU), et les **métadonnées de topologie**. Pipeline canonique : OpenTelemetry (instrumentation applicative ou *service mesh* Istio non intrusif) → Prometheus + Jaeger → estimation des α_{ij}, inférence et calcul des indicateurs de SLO. Le coût de collecte doit être intégré au budget (environ 558 mCPU et 936 Mo pour Prometheus+Jaeger, et 1 148 mCPU pour l'extraction périodique des dépendances).

Q10. Est-ce que l'autoscaling est différent de l'équilibrage de charge au niveau d'une API gateway ? Si oui, dans quelle mesure, si non, pourquoi ?

Oui, fondamentalement différents et complémentaires. L'autoscaling agit sur la **quantité de capacité** (combien d'instances) ; l'équilibrage agit sur la **direction du trafic** (vers quelle instance). Temporalités : longue et asynchrone (cold start) versus courte et synchrone (par requête). Ils ne sont pas substituables : l'équilibreur présuppose l'existence des instances (il ne crée pas de capacité), et l'autoscaling sans équilibrage laisserait des instances inutilisées. Le pipeline typique est Client → Load Balancer → Gateway → Pods, le HPA ajustant les pods. Dans le mémoire, l'équilibrage peut être traité comme une *hypothèse d'infrastructure* (répartition équitable vers les instances dimensionnées), ce qui concentre la contribution sur les $n_i(t)$.

Q11. Comment se fait l'allocation de ressources dans Kubernetes ?

En trois étages. *(i) Spécification* : `requests` (garantie, dénominateur du calcul HPA) et `limits` (plafond), avec request ≤ limit. *(ii) Placement* par le scheduler : filtrage des nœuds faisables $u \in \mathcal{U}_{\text{feasible}}$ vérifiant

$$\sum_{c \in \text{conteneurs}(p)} \text{request}_{c,r} \;\le\; \text{capacité}_r(u) - \sum_{p' \in \text{pods}(u)} \sum_{c} \text{request}_{p',r},$$

puis scoring $\sum_i w_i\, s_i(u,p)$. *(iii) Application et éviction* : cgroups et classes de QoS BestEffort $\prec$ Burstable $\prec$ Guaranteed. En surplomb : `ResourceQuota` (contrainte budgétaire agrégée, analogue à $\sum_i c_i n_i(t) \le B(t)$) et `LimitRange` (bornes par objet). Point clé : cette chaîne ne décide jamais $n_i(t)$; le HPA est un contrôleur séparé dont la décision est ensuite consommée par le scheduler.

Q12. Quelles sont les littératures qui permettent de traiter ce problème, et laquelle servira de baseline ?

Cinq familles structurent l'état de l'art : le **HPA** (baseline industrielle, réactif local) ; les approches **conscientes des dépendances** (Microscaler ICWS 2019, Erms ASPLOS 2023, Autothrottle NSDI 2024) ; les approches **prédictives/proactives** (Seer ASPLOS 2019, Autopilot EuroSys 2020, GRAF CoNEXT 2021) ; l'**apprentissage par renforcement** (FIRM OSDI 2020, Sinan ASPLOS 2021) ; et les **synthèses** (survey IEEE TNSM 2024, survey arXiv 2025) avec les avancées ST-GNN et *call-graph-aware* de 2025–2026. La **baseline académique** recommandée est **Erms** : il est conscient des dépendances (réduction des violations de SLA d'un facteur 5), mais réactif, ce qui laisse ouverte la dimension pro-active de la contribution. La **baseline industrielle** est le HPA, et **GRAF** sert de point de comparaison secondaire (proactif et GNN, mais centré sur le CPU). Positionnement : « Erms est conscient des dépendances mais réactif ; nous ajoutons la proactivité sans perdre la conscience du graphe ». Toutes les références doivent être vérifiées (auteurs, venue, pages, DOI) sur ACM DL ou IEEE Xplore avant inscription en bibliographie.